Infraestrutura de software
Guia de Relatorios

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo (3–5 linhas)

O que o enunciado pede e o que você entrega. Não explique a implementação aqui.

O objetivo é desenvolver a sincronização do embarque de passageiros em uma estação de metrô. O sistema deve respeitar a quantidade de assentos disponíveis e impedir a partida enquanto houver passageiros embarcando.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Executar em linux unix ou macos	OK**s**	Programa compilado e executado no Ubuntu pelo WSL.
Definir struct station e as quatro funções exigidas	OK	Verificação da estrutura e das funções no arquivo sync_car.c.
Utilizar mutex e variáveis de condição	OK	Verificação das chamadas pthread_mutex_* e pthread_cond_* no código.
Bloquear passageiros quando não tiver vaga	OK	O teste verifica se a quantidade de passageiros embarcados respeita o número de vagas disponíveis.
Aguardar a confirmação de embarque antes da partida	OK	O teste verifica se o metrô retorna antes das confirmações de embarque e apresenta erro caso isso aconteça.
Permitir partida com vagas restantes quando não houver mais passageiros	OK	Na execução realizada, o último metrô tinha 15 vagas e embarcou os 5 passageiros restantes.
Utilizar apenas um mutex por estação	OK	A estrutura contém somente o

		campo mutex como trava.
Não utilizar semáforos ou outras primitivas de sincronização na implementação	OK	Inspecionar sync_car.c e sync_car.h, que utiliza apenas mutex e variáveis de condição.

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado.

Compilar:

```
make clean
make
```

Executar (exemplo):

```
./sync_car
```

4. Arquitetura (resumida)

Uma linha por arquivo, só o que é específico do seu projeto.

Arquivos e responsabilidades:

1. Arquivo 1: `sync_car.c` → **Responsabilidade:** Define a estrutura da estação e implementa as quatro funções de sincronização.

2. Arquivo 2: `sync_car-runner.c` → **Responsabilidade:** Contém o teste do professor e a função `main`, incluindo diretamente sync_car.c

3. Arquivo 3: Makefile → **Responsabilidade:** Compila o programa, executa o teste e remove os arquivos gerados.

4. Arquivo 4: README.md → **Responsabilidade:** Descreve os arquivos, o ambiente e os comandos para compilar e testar.

5. Arquivo 5: evidencias.log → **Responsabilidade:**Registra os comandos e as saídas do terminal usados para comprovar a compilação e os testes.

6. Arquivo: sync_car.h → **Responsabilidade:** Define a estrutura da estação e declara as quatro funções.

Decisões técnicas:

1. Decisão: Utilizar um único mutex por estação → **Motivo:** Proteger os contadores compartilhados e atender ao enunciado → **Consequência:** Evita alterações simultâneas que poderiam reservar a mesma vaga

2. Decisão: Utilizar variáveis de condição → **Motivo:** Aguardar vagas e o término do embarque sem espera ocupada → **Consequência:** As threads dormem durante a espera, liberando o mutex.

3. Decisão: Separar reserva de vaga e confirmação de embarque → **Motivo**: Reservar um assento não significa que o passageiro já sentou → **Consequência:** Permite embarques simultâneos e impede a partida enquanto houver passageiros embarcando.

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Mutex e variáveis de condição	Utilizar um mutex para proteger os contadores e duas variáveis de condição para coordenar a espera por vagas e a conclusão do embarque.	Não houve troca de estratégia.
S2			
S3			

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do evidencias.log, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Remover os arquivos compilados com make clean para recompilar do zero.	Sucesso: executável e arquivos .o removidos.		04/11/2026 primeira etapa da validação.	evidencias.log: comando make clean e saída rm -f *.o sync_car.
2	S1	Compilar o programa pelo Makefile com make.	Sucesso: executável gerado, sem mensagens de erro.		04/11/2026 após a limpeza.	evidencias.log: comandos de compilação e ligação executados pelo GCC.
3	S1	Executar o teste do professor com ./sync_car	Sucesso: 100 passageiros embarcaram em quatro metrôs, respeitando as vagas disponíveis.		04/11/2026 após a compilação,	evidencias.log: embarques de 44, 14, 21 e 21 passageiros, com mensagem final Looks good!

Regra prática: se algo deu muito errado (bug que te travou por horas), a linha da tabela resume em 1 frase e aponta para o trecho do evidencias.log com o detalhe.

6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

6.1 Log automático (faz o trabalho pesado por você)

No início da sessão de trabalho, rode:

```
script -a evidencias.log
```

Isso grava automaticamente todos os comandos e saídas do terminal num arquivo, sem você precisar tirar print de cada linha. Comece a sessão com date, whoami e pwd, assim o log já tem timestamp, usuário e diretório sem esforço extra. **O evidencias.log deve ser enviado dentro do .tar**

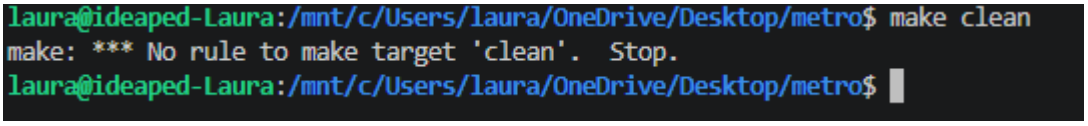
6.2 Prints

• Erro principal: o momento do bug mais relevante da semana.

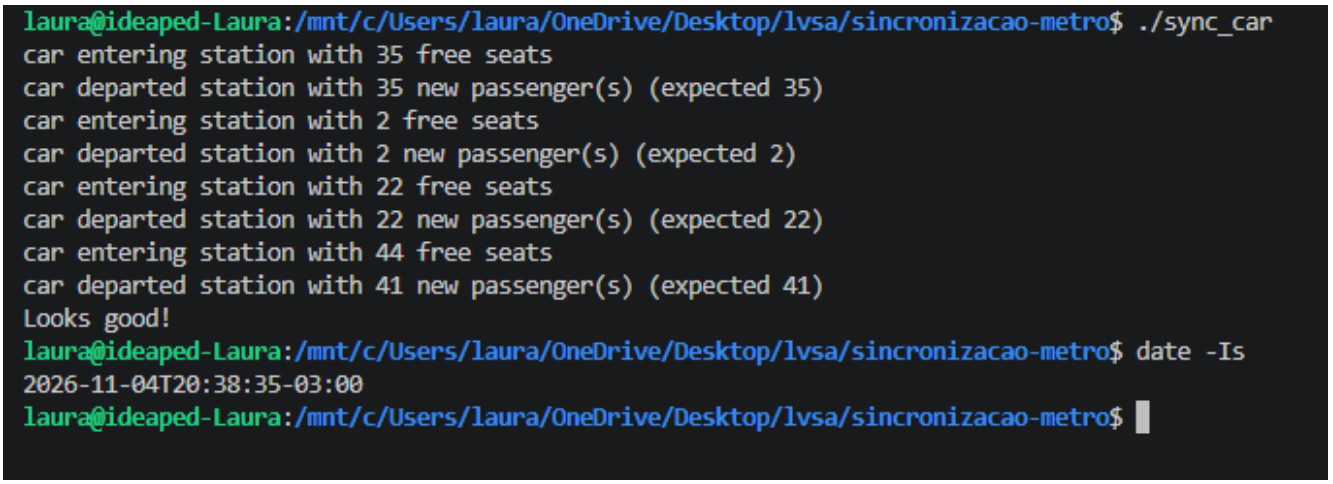
• Validação final: teste passando / saída correta.

Prints extras só se algo excepcional aconteceu e o log sozinho não explica, você precisa ter esse feeling, mas posso dar uma ajuda (ex.: comportamento visual, crash com stack trace longo).

Erro: make clean foi executado na pasta errada



Validação final: compilação pelo Makefile e execução do teste do professor concluída.



7. Uso de IA

Obrigatório preencher mesmo se a resposta for "não utilizei".

Onde usei IA (2–3 linhas):

Prompts principais (liste, sem história):

Utilizei a IA para entender o enunciado

Fiz o readme com o auxílio da IA

Pedi explicação sobre o mutex

O que validei manualmente e como:

Executei make clean e make no Ubuntu pelo WSL para verificar a compilação do projeto.

8. Reflexão Final (5–8 linhas)

O que você melhoraria com mais tempo, principais decisões técnicas e erros que evitaria na próxima. Tudo junto, sem subseções separadas.

Reflexão:

O principal cuidado foi fazer o metrô esperar todos os passageiros sentarem antes de sair e impedir que duas pessoas reservassem a mesma vaga. Na próxima atividade, conferiria a pasta antes de executar os comandos para evitar o erro que aconteceu. Também guardaria os registros e prints desde o começo para facilitar o relatório. Com mais tempo, faria mais testes.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	OK
Rodei os testes e evidencias.log foi gerado	OK
Tenho prints obrigatórios	OK
Testei pelo menos um caso limite e um caso inválido	OK
Preenchi a seção de uso de IA	OK
Revisei o relatório e removi frases genéricas / vazias	OK

10. Se eu tivesse mais 2 horas

Aplique as instruções abaixo de forma objetiva e verificável, evitando texto genérico. Escreva 5-10 linhas sobre o que você melhoraria (robustez, testes, warnings, performance, organização).

Adicionaria um teste em que um passageiro demora para sentar, verificando se o metrô espera, executaria os testes várias vezes para observar diferentes ordens de execução e definiria como tratar uma quantidade negativa de assentos e criaria um teste para esse caso.

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.